

Sumário

Resumo e Principais Tópicos.....	5
Contexto Inicial	5
Problemas Encontrados.....	5
Soluções Tradicionais e Seus Problemas	5
Containers como Solução.....	5
Dicas	6
Resumo e Principais Tópicos Responder às perguntas:.....	7
1. Por que Containers São Mais Leves que VMs?.....	7
2. Como os Containers Garantem Isolamento?	7
3. Como Funcionam sem Instalar um SO?	7
4. Divisão de Recursos (CPU, Memória, etc.)	8
Exemplo:.....	8
Conclusão	8
Documentações: Ubuntu Docker Docs Aprenda mais sobre Máquinas Virtuais com o Artigo VirtualBox e Máquinas Virtuais	9
Resumo: Docker e Virtualização com Containers.....	10
O que é o Docker?	10
Principais Benefícios do Docker	10
Como o Docker Funciona?	11
Casos de Uso do Docker.....	11
Dicas Práticas.....	11
Conclusão.....	12
Fluxo de Execução do Comando docker run	13
Resumo dos Comandos Docker Essenciais.....	16
1. Gerenciamento de Containers	16
2. Interação com Containers	16
3. Ciclo de Vida de um Container	17
4. Comparação: stop vs pause	17
5. Fluxo Prático	18
Para inspecionar containers:	18
Resumo: Executando um Container Docker com Aplicação Web	19
Principais Tópicos Abordados	19
Dicas Importantes.....	20
Conclusão: A aula demonstrou como executar, gerenciar e expor uma aplicação web em um container Docker, destacando flags essenciais (-d, -P, -p) e boas práticas.....	21

Resumo: O que são Imagens Docker e Como Funcionam?	21
Principais Tópicos Abordados	21
Dicas Importantes	22
Conclusão: Entender imagens e containers é essencial para dominar o Docker. Com esse conhecimento, você está pronto para criar e gerenciar seus próprios ambientes de forma eficiente! 🚀	22
Resumo: Criando sua Primeira Imagem Docker com Node.js	23
Principais Tópicos Abordados	23
Dicas Importantes	24
Próximos Passos	25
Conclusão: Você criou sua primeira imagem Docker! Com esse fluxo, você pode empacotar qualquer aplicação e compartilhá-la de forma consistente. 🎉	25
Resumo dos Principais Tópicos	26
Dicas	27
Comandos Docker e Persistência de Dados	28
Principais Tópicos	28
Dicas Importantes	29
Principais Comandos	29
Bind Mounts no Docker	30
Principais Tópicos	30
Dicas Importantes	31
Resumo: Volumes no Docker	33
Principais Tópicos	33
Dicas Importantes	34
tmpfs no Docker - Armazenamento Temporário em Memória	36
O que é tmpfs?	36
Como Usar tmpfs?	36
Exemplo Prático	37
Comparação: Bind Mount vs Volume vs tmpfs	38
Comandos Principais	38
Dicas Importantes	38
Comunicação entre Contêineres no Docker	40
O Problema da Comunicação	40
Solução: Redes Docker	40
Como Melhorar a Comunicação?	40
Passo a Passo: Testando Comunicação	41

Próximos Passos	42
Dicas Importantes.....	42
Comunicação Estável entre Contêineres Docker	42
Principais Tópicos	42
Dicas e Comandos Importantes	43
Diferença entre as Redes host e none no Docker	44
1. Rede host.....	44
2. Rede none.....	45
Resumo: Comparação entre host e none	45
Comandos úteis para redes no Docker	46
Comunicação entre Contêineres via Rede Docker	47
Passo a Passo	47
Pontos-Chave	48
Comandos Úteis para Debug.....	49
Conclusão.....	49
Nessa aula aprendemos:	49
Introdução ao Docker Compose.....	50
Passo a Passo	50
Próximos Passos.....	52
Por que Usar Docker Compose?	52
Docker Compose - Configuração e Execução	53
Principais Tópicos	53
Principais Comandos	54
Observação: O Docker Compose simplifica a execução de múltiplos containers com uma única configuração YAML.....	54
Configurando Dependências e Otimizando Logs no Docker Compose.....	54
Principais Tópicos	54
Principais Comandos	55
Observações Importantes	56
Conclusão: O Docker Compose simplifica a orquestração de múltiplos containers, mas requer atenção em dependências e modos de execução para garantir o funcionamento correto.,	56
No Windows , os volumes do Docker são armazenados em um caminho diferente devido à arquitetura do sistema e ao uso do Docker Desktop (que roda em uma máquina virtual Linux).	56
Resumo da Conversa sobre DevOps, Carreira e Mercado	58
 Principais Tópicos Abordados.....	58

 Principais Comandos e Ferramentas Citados	59
---	----

Resumo e Principais Tópicos

Contexto Inicial

- Sistemas modernos são compostos por múltiplas aplicações e ferramentas que interagem entre si.
- Exemplo: Um sistema com **Nginx** (load balancer), uma aplicação **Java** e uma aplicação **C# (.NET)**.

Problemas Encontrados

1. **Conflito de Portas:** Todas as aplicações precisam da **porta 80** simultaneamente.
2. **Gerenciamento de Versões:**
 - Difícil atualização/downgrade (ex: C# .NET 9, Java 17, Nginx 1.17.0).
 - Risco de quebrar dependências ao alterar versões.
3. **Alocação de Recursos:**
 - Como definir limites de CPU e memória para cada aplicação?
 - Ex: C# precisa de 100 millicores de CPU e 200MB de RAM.
4. **Manutenção a Longo Prazo:**
 - Complexidade em gerenciar portas, recursos e versões em um único ambiente.

Soluções Tradicionais e Seus Problemas

1. **Máquinas Físicas Dedicadas**
 - **Vantagem:** Isolamento completo (sem conflitos de portas ou recursos).
 - **Desvantagem:** Custo proibitivo (uma máquina por aplicação é inviável em escala).
2. **Máquinas Virtuais (VMs)**
 - Usam **hypervisor** para virtualizar sistemas operacionais.
 - **Vantagem:** Isolamento e controle de recursos.
 - **Desvantagem: Overhead** (cada VM roda um SO completo, consumindo mais recursos).

Containers como Solução

- **Diferença das VMs:**
 - Sem hypervisor ou SO virtualizado.

- Isolamento direto no nível do kernel do SO hospedeiro.
- **Vantagens:**
 - **Leveza:** Menos consumo de recursos (não precisa de um SO completo por app).
 - **Isolamento:** Aplicações rodam independentemente (sem conflitos).
 - **Portabilidade:** Fácil gerenciamento de versões e dependências.
- **Perguntas-chave:**
 - Como o container isola as aplicações sem um SO dedicado?
 - Como os recursos (CPU, memória) são alocados?

Conclusão e Próximos Passos

- Containers oferecem uma solução **eficiente e escalável** para os problemas citados.
- No próximo vídeo:
 - Como containers garantem isolamento.
 - Como funcionam sem um SO completo.
 - Divisão de recursos entre containers.

Dicas

1. **Para evitar conflitos de portas:** Use containers para isolar aplicações (cada uma pode "enxergar" a porta 80 internamente, mas mapear para portas diferentes no host).
2. **Gerenciamento de versões:** Utilize imagens de containers com versões específicas (ex: nginx:1.17.0, openjdk:17).
3. **Controle de recursos:** Defina limites de CPU/memória nos containers (ex: docker run --memory=200m --cpus=0.1).
4. **Escalabilidade:** Containers permitem deploy rápido e replicação (útil para load balancing).
5. **Ferramentas recomendadas:**
 - **Docker** (para desenvolvimento e testes).
 - **Kubernetes** (para orquestração em produção).

Próximo passo: Entender a arquitetura interna dos containers e como eles operam de forma leve e isolada. 🚀

Resumo e Principais Tópicos Responder às perguntas:

1. Por que **containers são mais leves** que máquinas virtuais (VMs)?
2. Como garantem **isolamento**?
3. Como funcionam **sem instalar um sistema operacional**?
4. Como é feita a **divisão de recursos** do sistema?

1. Por que Containers São Mais Leves que VMs?

- **Máquinas Virtuais (VMs):**
 - Exigem um **SO completo** para cada aplicação.
 - Usam **hypervisor**, criando overhead (consumo extra de CPU, memória e armazenamento).
- **Containers:**
 - Rodam como **processos isolados** diretamente no SO hospedeiro.
 - **Não virtualizam um SO** → menos consumo de recursos.
 - São mais **rápidos** para iniciar/parar e ocupam **menos espaço**.

2. Como os Containers Garantem Isolamento?

Utilizam **namespaces** do Linux para isolar diferentes aspectos do sistema:

- **PID namespace:** Isola processos (um container não enxerga os processos de outro).
- **NET namespace:** Isola interfaces de rede (cada container tem sua própria rede).
- **IPC namespace:** Isola comunicação entre processos (evita conflitos).
- **MNT namespace:** Isola o sistema de arquivos (cada container tem seu próprio filesystem).
- **UTS namespace:** Isola identificadores do sistema (hostname, kernel compartilhado).

👉 **Resultado:** Cada container age como um ambiente independente, mesmo rodando no mesmo host.

3. Como Funcionam sem Instalar um SO?

- Containers **não precisam de um SO completo** porque:

- Usam o **kernel do host** (via **UTS namespace**).
- Apenas **bibliotecas e dependências** da aplicação são empacotadas.
- **Exemplo:**
 - Se o host usa **Linux**, o container compartilha o mesmo kernel, mas com isolamento.
 - Não é necessário instalar um **Windows/Linux** dentro do container.

4. Divisão de Recursos (CPU, Memória, etc.)

Usam **cgroups (Control Groups)** para gerenciar recursos:

- **Limite de CPU:** Definir quantos núcleos ou porcentagem de CPU um container pode usar.
- **Limite de Memória:** Evitar que um container consuma toda a RAM.
- **Prioridade de E/S (I/O):** Controlar acesso a disco/rede.

Exemplo:

sh

```
docker run --cpus="0.5" --memory="512m" nginx # Limita a 0.5 CPU e 512MB RAM
```

Conclusão

- **Containers vs. VMs:**

Aspecto	Containers	Máquinas Virtuais (VMs)
Isolamento	Namespaces + cgroups	Hypervisor + SO completo
Consumo	Leve (processos do host)	Pesado (SO virtualizado)
Inicialização	Segundos	Minutos
Portabilidade	Alta (imagens autocontidas)	Média (depende do SO da VM)

- **Próximos passos:**
 - Instalação do **Docker** (Windows/Linux).

- Uso prático de containers.

Dicas

1. **Para otimizar recursos:** Use `--cpus` e `--memory` no Docker para evitar consumo excessivo.
2. **Isolamento seguro:** Namespaces garantem que um container não afete outros ou o host.
3. **Sem SO redundante:** Containers são eficientes porque **compartilham o kernel do host**.
4. **Ferramentas complementares:**
 - **Docker** (para criação e execução de containers).
 - **Kubernetes** (para orquestração em grande escala).

Próximo tema: Instalação e primeiros passos com Docker! 🐳

Documentações:

[Ubuntu | Docker Docs](#)

[Aprenda mais sobre Máquinas Virtuais com o Artigo VirtualBox e Máquinas Virtuais](#)

[Saiba como instalar e utilizar uma Máquina Virtual com o vídeo Máquina Virtual: O que é e como instalar](#)

Resumo: Docker e Virtualização com Containers

O que é o Docker?

- Plataforma que implementa **virtualização em nível de sistema operacional** usando **containers**.
- Permite empacotar aplicações com **código, runtime, bibliotecas e dependências** em unidades isoladas e portáteis.

Principais Benefícios do Docker

1. Isolamento de Contextos

- Cada container tem:
 - **Sistema de arquivos próprio** (isolado do host e de outros containers).
 - **Processos independentes** (via namespaces do Linux).
 - **Rede dedicada** (evita conflitos de portas).
- **Vantagem:**
 - Aplicações não interferem umas nas outras ou no sistema hospedeiro.
 - Maior segurança e estabilidade.

2. Versionamento de Aplicações

- **Imagens Docker:**
 - São **imutáveis** e autocontidas (tudo necessário para rodar a aplicação).
 - Construídas em **camadas** (otimiza reutilização e armazenamento).
- **Dockerfile:**
 - Arquivo de texto que define os passos para criar uma imagem.
 - Exemplo:

dockerfile

```
FROM python:3.8 # Imagem base
```

```
./app # Cópia código para o container
```

```
RUN pip install -r /app/requirements.txt # Instala dependências
```

```
CMD ["python", "/app/main.py"] # Comando de execução
```

- **Vantagens do versionamento:**

- Consistência entre ambientes (desenvolvimento, teste, produção).
- Facilidade para rollback (voltar para versões anteriores).

Como o Docker Funciona?

1. **Imagens:** Modelo estático (como um "molde") do container.
2. **Containers:** Instâncias em execução de uma imagem.
3. **Docker Engine:**
 - Gerencia containers (inicia, para, monitora recursos).
 - Usa **namespaces** e **cgroups** para isolamento e controle de recursos.

Comparativo: Docker vs. Virtualização Tradicional (VMs)

Característica	Docker (Containers)	Máquinas Virtuais (VMs)
Isolamento	Processos do host (via namespaces)	SO completo virtualizado
Consumo de Recursos	Leve (compartilha kernel)	Pesado (cada VM tem um SO)
Inicialização	Segundos	Minutos
Portabilidade	Alta (imagens únicas)	Média (depende do SO da VM)

Casos de Uso do Docker

- **Desenvolvimento:** Ambientes consistentes para toda a equipe.
- **CI/CD:** Integração contínua com testes isolados.
- **Microserviços:** Isolamento e escalabilidade de serviços.
- **Deploy em Nuvem:** Facilidade para rodar em qualquer cloud (AWS, Azure, GCP).

Dicas Práticas

1. **Otimize imagens:**
 - Use imagens oficiais (ex: python:3.8-slim).

- Remova arquivos desnecessários com `.dockerignore`.
 - 2. **Gerenciamento de recursos:**
 - Limite CPU/memória com `--cpus` e `--memory`.
 - 3. **Orquestração:**
 - Use **Kubernetes** ou **Docker Swarm** para gerenciar múltiplos containers.
-

Conclusão

O Docker revoluciona a entrega de software com:

- **Isolamento seguro** (sem overhead de VMs).
- **Versionamento confiável** (imagens imutáveis).
- **Portabilidade** ("rode em qualquer lugar").

Próximos passos:

- Instale o Docker ([Windows](#), [Linux](#)).
- Experimente comandos básicos (`docker run`, `docker build`).

✦ **Dúvidas?** Participe do fórum da Alura para discutir casos reais! 🚀

Fluxo de Execução do Comando docker run

Quando você executa `docker run [OPÇÕES] IMAGEM`, o Docker segue estas etapas em ordem:

1. Verificação da Imagem Local

- O Docker verifica se a **imagem especificada** (ex: `nginx:latest`) já existe no cache local (`docker images`).
- Se **não existir**, ele parte para o próximo passo.

2. da Imagem (Se Necessário)

- Se a imagem não estiver localmente, o Docker a **baixa do registry padrão** (Docker Hub) ou de um registry customizado.
- Exemplo:

```
bash
```

```
docker pull nginx:latest # (Etapa implícita no `docker run` se a imagem não existir)
```

3. Criação do Container

- O Docker **cria um novo container** a partir da imagem, configurando:
 - **Filesystem isolado** (usando as camadas da imagem + uma camada gravável temporária).
 - **Namespaces** (isolamento de processos, rede, etc.).
 - **Cgroups** (limites de CPU/memória, se definidos).

4. Configuração da Rede

- O Docker **conecta o container a uma rede**:
 - Se nenhuma rede for especificada, usa a rede padrão (`bridge`).
 - Atribui um **IP interno** e mapeia portas (se usado `-p HOST:CONTAINER`).

5. Configuração de Variáveis de Ambiente (Se Aplicável)

- Se houver variáveis definidas (via `-e` ou no `Dockerfile`), elas são injetadas no container.

```
bash
```

```
docker run -e "VAR=valor" nginx
```

6. Montagem de Volumes (Se Aplicável)

- Se volumes ou bind mounts forem especificados (-v ou --mount), o Docker os vincula ao container.

```
bash
```

```
docker run -v /pasta/local:/pasta/container nginx
```

7. Execução do Comando Principal

- O Docker **inicia o processo principal** do container (definido por CMD ou ENTRYPOINT no Dockerfile).
- Se um comando customizado for passado no docker run, ele substitui o CMD padrão.

```
bash
```

```
docker run nginx echo "Olá, Docker!" # Substitui o CMD do Nginx
```

8. Gerenciamento do Ciclo de Vida

- O container entra em estado **"Running"**.
- Se o processo principal terminar, o container **para** (a menos que seja usado -d para rodar em segundo plano).

Exemplo Completo

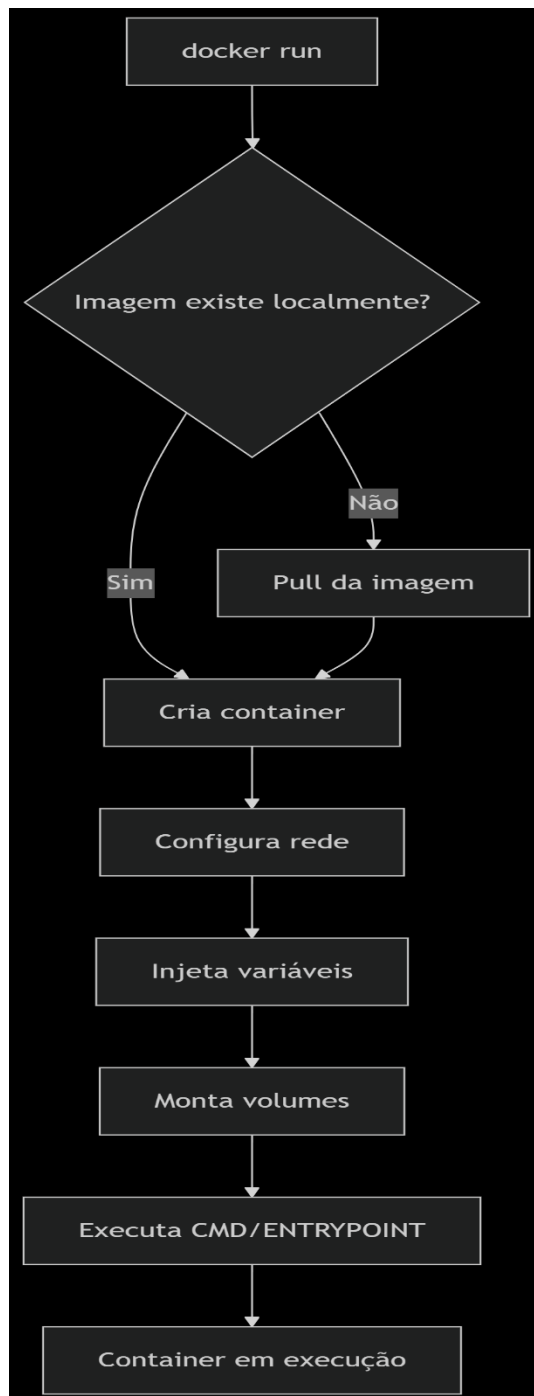
```
bash
```

```
docker run -d --name meu-nginx -p 8080:80 -e "ENV=prod" -v  
./dados:/usr/share/nginx/html nginx:latest
```

Ordem de execução:

1. Verifica se nginx:latest existe localmente → Se não, baixa do Docker Hub.
2. Cria o container meu-nginx com isolamento.
3. Conecta-o à rede bridge e mapeia a porta 8080 do host para 80 do container.
4. Injeta a variável ENV=prod.

5. Monta o volume ./dados no container.
6. Roda o comando padrão do Nginx (CMD ["nginx", "-g", "daemon off;"]).
7. Mantém o container ativo em segundo plano (-d).



Resumo dos Comandos Docker Essenciais

1. Gerenciamento de Containers

Comando	Descrição	Exemplo
<code>docker ps</code>	Lista containers em execução	<code>docker ps</code>
<code>docker ps -a</code>	Lista todos containers (incluindo parados)	<code>docker ps -a</code>
<code>docker stop</code>	Para um container (envia sinal SIGTERM)	<code>docker stop <ID/NOME></code>
<code>docker stop -t=0</code>	Para imediatamente (SIGKILL)	<code>docker stop -t=0 <ID></code>
<code>docker start</code>	Reinicia um container parado	<code>docker start <ID></code>
<code>docker pause</code>	Pausa um container (congela processos)	<code>docker pause <ID></code>
<code>docker unpause</code>	Despausa um container	<code>docker unpause <ID></code>
<code>docker rm</code>	Remove um container parado	<code>docker rm <ID></code>
<code>docker rm -f</code>	Remove um container em execução (forçado)	<code>docker rm -f <ID></code>

2. Interação com Containers

Comando	Descrição	Exemplo
<code>docker exec -it</code>	Executa um comando dentro do container (modo interativo)	<code>docker exec -it <ID> bash</code>
<code>docker run -it</code>	Cria e já acessa o container (terminal interativo)	<code>docker run -it ubuntu bash</code>

👉 **Observação:**

- Se o **processo principal** (ex: bash) for encerrado, o container **para**.
- Use sleep ou processos em segundo plano (-d) para manter o container ativo.

3. Ciclo de Vida de um Container

1. Criação:

bash

docker run -d --name meu-container ubuntu sleep 1d

- Cria um container em segundo plano (-d) com o comando sleep 1d.

2. Acesso:

bash

docker exec -it meu-container bash

- Acessa o terminal do container.

3. Persistência de Dados:

- Arquivos criados **dentro** do container são **efêmeros** (perdidos ao remover o container).
- Solução futura: **Volumes Docker** (persistência).

4. Encerramento:

- Se o processo principal (sleep) terminar, o container **para**.
- Se você sair do bash (Ctrl+D), o container **continua** (pois sleep ainda está ativo).

4. Comparação: stop vs pause

Ação

Efeito

docker stop	Encerra processos (SIGTERM/SIGKILL) → Reinicia PID ao usar start .
-------------	---

Ação	Efeito
docker pause	Congela processos (não encerra) → Mantém estado exato ao despausar.

5. Fluxo Prático

Diagram

Code

Mermaid rendering failed.

Dicas Importantes

1. Para containers efêmeros:

- Use `docker run --rm` para **auto-remover** o container ao parar.

bash

```
docker run --rm -it ubuntu bash
```

2. Para evitar conflitos de nomes:

- Use `--name` para identificar containers facilmente.

bash

```
docker run --name meu-app -d nginx
```

Para inspecionar containers:

- Use `docker inspect` para ver detalhes (redes, volumes, etc.).

bash

```
docker inspect <ID>
```

Próximos Passos

- **Persistência de dados:** Como usar volumes e bind mounts.
- **Dockerfile:** Como criar imagens personalizadas.
- **Orquestração:** Introdução ao docker-compose.

📌 **Key Takeaway:** Containers são **leves, isolados e efêmeros** – gerencie-os com os comandos acima! 🐳

New chat

Resumo: Executando um Container Docker com Aplicação Web

Principais Tópicos Abordados

1. Exemplo Prático com Aplicação Web

- Utilização da imagem não oficial dockersamples/static-site do Docker Hub para demonstração.
- Diferença entre imagens oficiais (com verificação) e não oficiais.

2. Execução do Container em Segundo Plano

- Uso da flag -d (detached) para rodar o container sem travar o terminal:

bash

```
docker run -d dockersamples/static-site
```

3. Verificação do Container em Execução

- Comando docker ps mostra o container ativo, expondo as portas **80** e **443**.

4. Problema de Acesso à Aplicação

- O container roda em um namespace de rede isolado, então localhost:80 não funciona diretamente.

5. Mapeamento de Portas Automático

- Flag -P (maiúsculo) mapeia automaticamente as portas do container para portas aleatórias no host:

bash

```
docker run -d -P dockersamples/static-site
```

- Verificação das portas mapeadas com `docker port <ID_CONTAINER>`.

6. Mapeamento de Portas Manual

- Flag -p (minúsculo) permite definir manualmente a porta do host e do container:

bash

```
docker run -d -p 8080:80 dockersamples/static-site
```

- Acesso via `http://localhost:8080`.

7. Remoção Forçada do Container

- Comando para parar e remover o container em uma única etapa:

bash

```
docker rm <ID_CONTAINER> --force
```

Dicas Importantes

✅ **Prefira imagens oficiais** sempre que possível para maior segurança e confiabilidade.

- O Docker Hub é um grande repositório de imagens que podemos utilizar;
- A base dos containers são as imagens;

✅ **Use -d** para executar containers em segundo plano e liberar o terminal.

✅ **Mapeie portas** com -P (automático) ou -p HOST:CONTAINER (manual) para expor serviços.

✅ **Verifique portas** com `docker port <ID>` ou `docker ps` para ver o mapeamento.

✅ **Remova containers parados** com `docker rm --force` para limpar recursos não

utilizados.

✓ **Acesse a aplicação** no navegador usando a porta mapeada (ex: localhost:8080).

Conclusão: A aula demonstrou como executar, gerenciar e expor uma aplicação web em um container Docker, destacando flags essenciais (-d, -P, -p) e boas práticas.

Resumo: O que são Imagens Docker e Como Funcionam?

Principais Tópicos Abordados

1. Definição de Imagens Docker

- Uma imagem é um **conjunto de camadas imutáveis (read-only)** que, quando combinadas, formam a base para criar containers.
- Cada camada tem um **ID único** e contém alterações específicas em relação à camada anterior.

2. Estrutura das Imagens

- **Camadas (Layers):** Arquivos empilhados que representam mudanças incrementais (instalação de pacotes, cópias de arquivos, configurações).
- **Reutilização:** Camadas são compartilhadas entre imagens para otimizar espaço e desempenho.

3. Comandos para Analisar Imagens

- `docker images` ou `docker image ls`: Lista imagens baixadas no sistema.
- `docker inspect <ID_IMAGEM>`: Exibe informações detalhadas (ID, data de criação, configurações).
- `docker history <ID_IMAGEM>`: Mostra as camadas que compõem a imagem e seus tamanhos.

4. Como uma Imagem Vira um Container?

- Ao executar `docker run`, o Docker:
 1. Baixa as camadas necessárias (se não estiverem no host).
 2. Adiciona uma **camada temporária de leitura/escrita (read-write)** em cima da imagem (read-only).
- **Dados escritos no container** são armazenados nessa camada temporária e **perdidos quando o container é removido**.

5. Por que Containers São Leves?

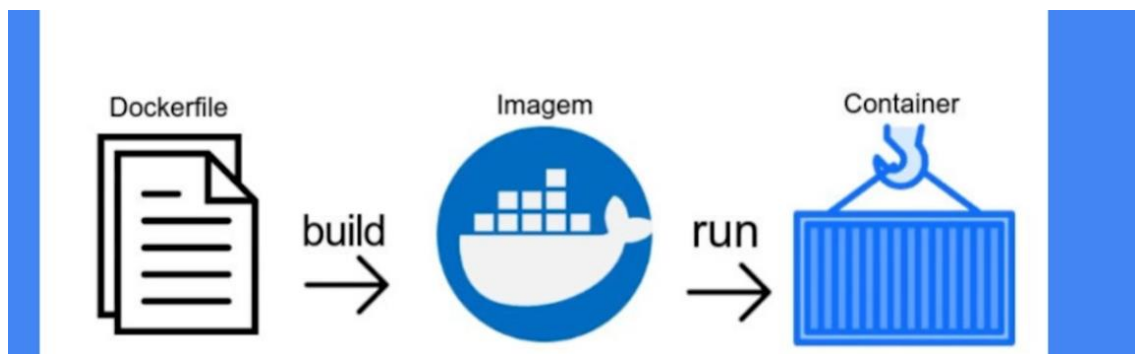
- **Compartilhamento de camadas:** Múltiplos containers podem usar a mesma imagem base, adicionando apenas uma camada fina de escrita.
- **Eficiência:** A imagem original (read-only) não é duplicada, apenas reutilizada.

Dicas Importantes

- ✓ **Use docker history** para entender como uma imagem foi construída (ordem das camadas e comandos usados).
- ✓ **Imagens são imutáveis:** Alterações em containers não afetam a imagem original.
- ✓ **Containers efêmeros:** Dados não persistidos em volumes são perdidos ao remover o container.
- ✓ **Otimize suas imagens:** Camadas menores e bem organizadas melhoram desempenho e reduz tamanho.

Próximos Passos

- **Criação de Imagens Personalizadas:** Aprenderemos a usar o Dockerfile para definir nossas próprias imagens.
- **Persistência de Dados:** Como usar volumes para armazenar informações permanentemente.



Conclusão: Entender imagens e containers é essencial para dominar o Docker. Com esse conhecimento, você está pronto para criar e gerenciar seus próprios ambientes de forma eficiente!



Resumo: Criando sua Primeira Imagem Docker com Node.js

Principais Tópicos Abordados

1. Objetivo

- Criar uma imagem Docker personalizada para uma aplicação Node.js simples (exibe "Eu amo Docker!").
- **Fluxo básico:**
 - Escrever um Dockerfile → Construir a imagem (docker build) → Executar o container (docker run).

2. Arquivo Dockerfile

- **Base da imagem:** Usar a imagem oficial do Node.js (versão 14):

dockerfile

FROM node:14

- **Diretório de trabalho:** Definir /app-node como pasta padrão no container:

dockerfile

WORKDIR /app-node

- **Copiar arquivos:** Transferir o código do host para o container:

dockerfile

..

- **Instalar dependências:** Executar npm install durante a construção da imagem:

dockerfile

RUN npm install

- **Ponto de entrada:** Iniciar a aplicação com npm start quando o container rodar:

dockerfile

```
ENTRYPOINT ["npm", "start"]
```

3. Construindo a Imagem

- Comando para criar a imagem (a partir do diretório do projeto):

bash

```
docker build -t danielartini/app-node:1.0 .
```

- -t: Define o nome e a tag da imagem (ex: usuario/nome-da-imagem:versao).

4. Executando o Container

- Mapear a porta do host (8081) para a porta do container (3000):

bash

```
docker run -d -p 8081:3000 danielartini/app-node:1.0
```

- Acessar a aplicação no navegador: <http://localhost:8081>.

5. Documentação e Melhores Práticas

- Referência à [documentação oficial do Docker](#) para sintaxe do Dockerfile.
- Instruções úteis não usadas aqui: ENV (variáveis de ambiente), EXPOSE (expor portas), VOLUME (persistência de dados).

Dicas Importantes

- ✓ **Use imagens oficiais** (ex: node:14) como base para confiabilidade.
- ✓ **Defina WORKDIR** para evitar caminhos absolutos repetidos.
- ✓ **Ordene instruções do Dockerfile** do menos para o mais frequente alterado (otimiza cache de build).
- ✓ **Explicit portas:** Adicione EXPOSE 3000 no Dockerfile para documentar a porta usada pela aplicação.
- ✓ **Versionamento:** Sempre use tags (ex: :1.0) para facilitar rollbacks e rastreabilidade.

Problema Comum e Solução

- **Como saber a porta da aplicação no container?**
 - Verifique o código-fonte (ex: `app.listen(3000)` no `index.js`) ou use EXPOSE no Dockerfile.
 - Se a aplicação não responder, verifique logs do container:

bash

`docker logs <ID_CONTAINER>`

Próximos Passos

- **Melhorar o Dockerfile:**
 - Adicionar EXPOSE 3000.
 - Usar variáveis de ambiente (ENV) para configurações flexíveis.
- **Persistir dados:** Aprender sobre volumes Docker para dados não efêmeros.

Conclusão: Você criou sua primeira imagem Docker! Com esse fluxo, você pode empacotar qualquer aplicação e compartilhá-la de forma consistente. 🚀

<https://docs.docker.com/engine/reference/builder/>

Comandos:

`docker build -t danielartine/app-node:1 .`

`docker images`

`docker run -d -p 8081:3000 danielartine/app-node:1.0`

Resumo dos Principais Tópicos

1. Parando Containers em Massa

- Comando para parar todos os containers em execução:

bash

```
docker stop $(docker container ls -q)
```

- A flag -q retorna apenas os IDs dos containers.

2. Expondo Portas no Dockerfile

- A instrução EXPOSE documenta a porta em que a aplicação está rodando dentro do container.
- Exemplo:

dockerfile

```
EXPOSE 3000
```

- Facilita para outros usuários saberem qual porta mapear.

3. Trabalhando com Variáveis de Ambiente

- **ARG vs. ENV:**
 - ARG: Define variáveis durante o **build** da imagem (ex.: ARG PORT_BUILD=6000).
 - ENV: Define variáveis que persistem no **container em execução** (ex.: ENV PORT=\$PORT_BUILD).
- Uso no index.js:

javascript

```
app.listen(process.env.PORT, ...)
```

4. Build e Execução com Variáveis

- Build da imagem com nova versão:

bash

```
docker build -t danielartine/app-node:1.2 .
```

- Execução com mapeamento de porta:

```
bash
```

```
docker run -p 9090:6000 -d danielartine/app-node:1.2
```

5. Próximos Passos

- Deixar a imagem mais parametrizável e semântica.
- Preparar para deploy no Docker Hub.

Nesta aula do curso "Docker: criando e gerenciando containers", o foco foi na criação e aprimoramento de imagens Docker, especificamente no uso do Dockerfile. Os principais pontos abordados foram:

1. **Documentação da Porta:** Aprendeu-se a usar a instrução EXPOSE no Dockerfile para documentar a porta em que a aplicação está rodando, facilitando o entendimento para outros usuários.
2. **Uso de Variáveis de Ambiente:** Foi introduzido o conceito de variáveis de ambiente, utilizando ARG para definir valores durante a construção da imagem e ENV para definir variáveis que estarão disponíveis dentro do container em execução.
3. **Criação de Imagens:** A aula demonstrou como construir novas versões da imagem com o comando docker build, incluindo a definição de portas e variáveis de ambiente.
4. **Execução de Containers:** Foi mostrado como executar containers com o comando docker run, incluindo o mapeamento de portas entre a máquina host e o container.
5. **Próximos Passos:** A aula concluiu com a promessa de abordar o deploy da imagem no Docker Hub nas próximas aulas.

Esses conceitos ajudam a tornar as imagens Docker mais robustas e fáceis de usar.

Dicas

- ✓ **Use EXPOSE** para documentar portas no Dockerfile, facilitando o uso por outros desenvolvedores.
- ✓ **Prefira variáveis de ambiente (ENV)** para configurações dinâmicas, como portas.

- ✓ **Use ARG** para valores temporários durante o build da imagem.
- ✓ **Combine -p com EXPOSE** no docker run para mapeamento explícito de portas.
- ✓ **Teste sempre** após mudanças no Dockerfile para garantir que as variáveis e portas funcionam como esperado.

Comandos Docker e Persistência de Dados

Principais Tópicos

1. Remoção de Containers e Imagens

- `docker container rm $(docker container ls -aq)` → Remove **todos** os containers (parados e em execução).
- `docker rmi $(docker image ls -aq)` → Remove **todas** as imagens.
- `--force` → Força a remoção de imagens com conflitos.

2. Verificação de Containers e Imagens

- `docker ps` → Lista containers em execução.
- `docker ps -a` → Lista **todos** os containers (incluindo parados).
- `docker images` → Lista imagens disponíveis.

3. Tamanho do Container

- `docker ps -s` → Mostra o tamanho real (camada de read-write) e virtual (imagem base + modificações).
- `docker history <imagem>` → Exibe as camadas que compõem uma imagem.

4. Persistência de Dados

- Containers têm uma **camada temporária (read-write)** que é perdida ao removê-los.
- **3 formas de persistir dados:**
 - **Bind Mount** → Vincula um diretório do host ao container.
 - **Volume** → Armazenamento gerenciado pelo Docker.
 - **tmpfs mount** → Armazenamento temporário em memória (não persiste após reinício).

Dicas Importantes

- ✓ Use `--force` se uma imagem não puder ser removida devido a conflitos.
- ✓ O **tamanho virtual** de um container é a soma da imagem base + modificações.
- ✓ Dados em containers **não são persistentes** por padrão – use **volumes** ou **bind mounts** para armazenamento duradouro.

Principais Comandos

Comando	Descrição
<code>docker container rm \$(docker container ls -aq)</code>	Remove todos os containers
<code>docker rmi \$(docker image ls -aq) --force</code>	Remove todas as imagens (forçado)
<code>docker ps -s</code>	Mostra tamanho do container
<code>docker history <imagem></code>	Exibe camadas da imagem
<code>docker run -it ubuntu bash</code>	Inicia um container Ubuntu interativo
<code>exit</code>	Sai do terminal interativo do container

Próximo Passo: Aprofundar em **volumes** e **bind mounts** para persistência de dados! 🚀

Bind Mounts no Docker

Principais Tópicos

1. O que é um Bind Mount?

- **Ligação entre um diretório do host (sistema operacional) e um diretório dentro do container.**
- Permite persistir dados mesmo após a remoção do container.

2. Criando um Bind Mount

- **Método 1: Flag -v (mais simples)**

bash

```
docker run -it -v /caminho/no/host:/caminho/no/container ubuntu bash
```

- Exemplo:

bash

```
docker run -it -v /home/usuario/volume-docker:/app ubuntu bash
```

→ Tudo em /app no container será salvo em /home/usuario/volume-docker no host.

- **Método 2: Flag --mount (recomendado pela Docker)**

bash

```
docker run -it --mount type=bind,source=/caminho/no/host,target=/caminho/no/container  
ubuntu bash
```

- Exemplo:

bash

```
docker run -it --mount type=bind,source=/home/usuario/volume-docker,target=/app  
ubuntu bash
```

- **Vantagem:** Mais explícito e semântico.

- **Cuidado:** Se o caminho no host não existir, o Docker retornará um erro.

3. Funcionamento Prático

- Arquivos criados no diretório montado (ex.: /app) **persistem no host**.
- Se o container for removido e outro for criado com o mesmo bind mount, os dados **continuam disponíveis**.
- **Exemplo:**

bash

touch /app/arquivo.txt # No container

→ O arquivo aparecerá em /home/usuario/volume-docker/arquivo.txt no host.

4. Limitações do Bind Mount

- **Dependência do caminho no host:** Se o diretório for movido, renomeado ou excluído, o container não funcionará corretamente.
- **Problemas de permissão:** O container pode não ter acesso ao diretório no host.

5. Alternativa Recomendada: Volumes Docker

- **Gerenciados pelo Docker** (não dependem de caminhos absolutos no host).
- Mais seguro e portátil (será abordado no próximo tópico).

Dicas Importantes

- ✓ **Prefira** --mount em vez de -v para maior clareza (recomendação oficial).
- ✓ **Verifique o caminho no host** antes de usar bind mounts para evitar erros.
- ✓ **Use volumes Docker** se precisar de portabilidade ou evitar dependência do filesystem do host.

Principais Comandos

Comando	Descrição
docker run -it -v /host:/container ubuntu bash	Cria um bind mount com -v

Comando	Descrição
<code>docker run -it --mount type=bind,source=/host,target=/container ubuntu bash</code>	Cria um bind mount com --mount
<code>touch /caminho/no/container/arquivo.txt</code>	Cria um arquivo persistente no bind mount
<code>ls /caminho/no/host</code>	Verifica os arquivos persistidos no host

Próximo Passo: Explorar **volumes Docker** para uma solução mais robusta de persistência! 🚀

Resumo: Volumes no Docker

Principais Tópicos

1. O que são Volumes Docker?

- **Área gerenciada pelo Docker** no sistema de arquivos do host.
- **Mais seguro e recomendado para produção** (evita problemas de permissão e dependência de caminhos absolutos).
- Dados são armazenados em `/var/lib/docker/volumes/`.

2. Criando e Gerenciando Volumes

- **Criar um volume:**

bash

```
docker volume create meu-volume
```

- **Listar volumes:**

bash

```
docker volume ls
```

- **Inspecionar um volume:**

bash

```
docker volume inspect meu-volume
```

- **Remover um volume:**

bash

```
docker volume rm meu-volume
```

- **Remover volumes não utilizados:**

bash

```
docker volume prune
```

3. Usando Volumes em Containers

- **Método 1: Flag -v**

bash

```
docker run -it -v meu-volume:/app ubuntu bash
```

- Se o volume não existir, o **Docker o cria automaticamente**.

- **Método 2: Flag --mount (recomendado)**

bash

```
docker run -it --mount source=meu-volume,target=/app ubuntu bash
```

- Mais explícito e semântico.
- **Se o volume não existir, o Docker o cria automaticamente.**

4. Onde os Dados são Armazenados?

- **Local padrão:**

```
bash
```

```
/var/lib/docker/volumes/<nome-do-volume>/_data/
```

- **Como acessar:**

```
bash
```

```
sudo su
```

```
cd /var/lib/docker/volumes/meu-volume/_data/
```

```
ls
```

5. Vantagens dos Volumes sobre Bind Mounts

- **Gerenciamento pelo Docker** (não depende de caminhos no host).
- **Mais seguro** (evita acesso acidental a arquivos do host).
- **Portabilidade** (facilita migração entre ambientes).
- **Backup e recuperação simplificados.**

Dicas Importantes

✓ **Prefira** `--mount` para maior clareza (recomendação oficial).

✓ **Não é necessário criar o volume antes** – o Docker cria automaticamente se ele não existir.

✓ **Evite manipular** `/var/lib/docker/volumes/` **manualmente** – use comandos do Docker (`volume rm`, `prune`).

✓ **Use volumes em produção** para maior segurança e gerenciamento simplificado.

Alternativa: Usar `docker volume inspect`

Se você quer descobrir o caminho exato de um volume **sem navegar manualmente**, use:

```
bash
```

```
docker volume inspect nome-do-volume
```

Principais Comandos

Comando	Descrição
<code>docker volume create <nome></code>	Cria um novo volume
<code>docker volume ls</code>	Lista volumes existentes
<code>docker volume inspect <nome></code>	Mostra detalhes do volume
<code>docker run -v <volume>:<caminho-no-container></code>	Monta um volume no container
<code>docker run --mount source=<volume>,target=<caminho></code>	Método recomendado para montar volumes
<code>docker volume rm <nome></code>	Remove um volume
<code>docker volume prune</code>	Remove volumes não utilizados

Próximo Passo: Explorar **tmpfs mounts** para armazenamento temporário em memória!



tmpfs no Docker - Armazenamento Temporário em Memória

O que é tmpfs?

- **Armazenamento temporário em memória RAM** (não persiste após a parada do container).
- **Exclusivo para Linux** (não funciona em Windows/Mac sem configurações extras).
- **Dados sensíveis:** Ideal para informações que **não devem ser persistidas** no filesystem (ex.: tokens, senhas temporárias).

Como Usar tmpfs?

Método 1: Flag --tmpfs

bash

```
docker run -it --tmpfs=/app ubuntu bash
```

- **Funcionamento:**
 - Cria a pasta /app no container **em memória RAM**.
 - Arquivos em /app são **perdidos** quando o container é removido.

Método 2: Flag --mount (Recomendado)

bash

```
docker run -it --mount type=tmpfs,destination=/app ubuntu bash
```

- **Vantagem:** Mais explícito e alinhado com a sintaxe de outros mounts.

Características do tmpfs

Feature	Detalhe
Persistência	Dados são apagados quando o container é removido.

Feature	Detalhe
Performance	Extremamente rápido (armazenamento em RAM).
Segurança	Evita vazamento de dados sensíveis (não escreve em disco).
Pastas verdes no ls	Pastas com tmpfs aparecem em verde (indicando armazenamento temporário).

Exemplo Prático

1. Criar container com tmpfs:

```
bash
```

```
docker run -it --tmpfs=/app ubuntu bash
```

2. Criar arquivo temporário:

```
bash
```

```
touch /app/arquivo-temporario.txt
```

3. Verificar conteúdo:

```
bash
```

```
ls /app # Mostrará o arquivo.
```

4. Parar container e recriar:

```
bash
```

```
exit
```

```
docker run -it --tmpfs=/app ubuntu bash
```

```
ls /app # Vazio (dados perdidos).
```

Quando Usar tmpfs?

- **Dados efêmeros:** Cache, sessões temporárias, logs não críticos.
- **Segurança:** Evitar persistência de senhas, chaves API ou tokens.
- **Performance:** Aplicações que exigem acesso ultrarrápido a dados temporários.

Comparação: Bind Mount vs Volume vs tmpfs

Tipo	Armazenamento	Persistência	Uso Recomendado
Bind Mount	Diretório do host	Sim	Configurações ou desenvolvimento local.
Volume	Gerenciado pelo Docker	Sim	Produção (dados persistentes).
tmpfs	Memória RAM (temporário)	Não	Dados sensíveis ou temporários.

Comandos Principais

Comando	Descrição
<code>docker run --tmpfs=/caminho</code>	Cria tmpfs no caminho especificado.
<code>docker run --mount type=tmpfs,destination=/caminho</code>	Método recomendado para tmpfs.

Dicas Importantes

- ⚠ **Linux apenas:** tmpfs não funciona nativamente em Windows/Mac.
- 🚀 **Performance:** Use para dados que precisam de acesso rápido e não requerem persistência.
- 🔒 **Segurança:** Ideal para evitar vazamento de informações confidenciais.

Próximo Passo: Aplicar esses conceitos em um projeto real com múltiplas camadas de persistência! 🚀

Comunicação entre Contêineres no Docker

O Problema da Comunicação

Contêineres são **isolados por padrão**, mas sistemas complexos exigem comunicação entre eles (ex.: back-end ↔ banco de dados).

Como conectar contêineres de forma eficiente e estável?

Solução: Redes Docker

1. Rede Padrão bridge

- **Criada automaticamente** pelo Docker (docker0).
- Contêineres sem rede definida são **conectados a ela**.
- **Comunicação via IP:**
- **Comunicação via IP:**

bash

`docker inspect <ID_CONTÊINER>` # Verifica o IP (ex.: 172.17.0.2)

`ping 172.17.0.2` # Teste de comunicação (requer `iputils-ping` instalado)

- **Problema:** IPs mudam se o contêiner for recriado → **Solução instável.**

2. Listando Redes Existentes

bash

`docker network ls`

Saída:

text

NETWORK ID	NAME	DRIVER	SCOPE
------------	------	--------	-------

80a1db0b6238	bridge	bridge	local # Rede padrão
--------------	--------	--------	---------------------

df26e341d36e	host	host	local # Usa a rede do host
--------------	------	------	----------------------------

6ef79c7aa3d6	none	null	local # Sem rede
--------------	------	------	------------------

Como Melhorar a Comunicação?

a) Usar Nomes (DNS Interno)

- Contêineres na **mesma rede** podem se comunicar via **nome** (não apenas IP).
- Exemplo:


```
bash
```

```
docker run --name meu-container -it ubuntu bash
```

- Outro contêiner pode acessá-lo via ping meu-container.

b) Criar Redes Personalizadas

- **Mais controle** sobre configurações de rede.
- **Melhor isolamento** e segurança.
- Comando:

```
bash
```

```
docker network create minha-rede
```

```
docker run --network minha-rede -it ubuntu bash
```

Passo a Passo: Testando Comunicação

1. **Crie dois contêineres na rede padrão bridge:**

```
bash
```

```
docker run -it --name container1 ubuntu bash
```

```
docker run -it --name container2 ubuntu bash
```

2. **No container1, instale o ping:**

```
bash
```

```
apt-get update && apt-get install -y iputils-ping
```

3. **Teste a comunicação com container2:**

```
bash
```

```
ping container2 # Funciona se estiverem na mesma rede!
```

Comparação: Tipos de Rede Docker

Tipo	Descrição	Uso
bridge	Rede padrão (NAT interna)	Comunicação básica entre contêineres.

Tipo	Descrição	Uso
host	Usa a rede do host diretamente	Alta performance (sem isolamento).
none	Sem rede	Casos extremos de isolamento.
Custom	Rede criada pelo usuário	Controle total (recomendado para produção).

Próximos Passos

- **Redes personalizadas:** Como criar e gerenciar redes específicas para aplicações.
- **DNS automático:** Comunicação estável usando nomes de contêineres.
- **Segurança:** Isolar redes para serviços críticos (ex.: banco de dados).

Dicas Importantes

- ♦ **Evite depender de IPs** → Use **nomes de contêineres** ou redes customizadas.
- ♦ **Instale ferramentas de rede** (ping, curl) em imagens mínimas (ex.: ubuntu) para testes.
- ♦ **Produção:** Sempre use **redes personalizadas** para maior controle e segurança.

Comando útil:

```
bash
```

```
docker network inspect bridge # Detalhes da rede padrão
```

No próximo vídeo: **Como criar e gerenciar redes Docker personalizadas!** 🚀

Comunicação Estável entre Contêineres Docker

Principais Tópicos

1. Problema com IPs dinâmicos

- Contêineres têm IPs atribuídos dinamicamente, o que dificulta a comunicação estável.

2. Solução: Nomear contêineres

- Usar `--name` para definir um nome fixo ao criar um contêiner:

```
docker run -it --name ubuntu1 ubuntu bash
```

3. Criando uma rede personalizada (user-defined bridge)

- Redes padrão (bridge, host, none) não resolvem DNS automaticamente.
- Criar uma rede bridge personalizada:

```
docker network create --driver bridge minha-bridge
```

4. Conectando contêineres à rede

- Usar `--network` para associar contêineres à rede criada:

```
docker run -it --name ubuntu1 --network minha-bridge ubuntu bash
```

```
docker run -d --name pong --network minha-bridge ubuntu sleep 1d
```

5. Comunicação via hostname

- Contêineres na mesma rede podem se comunicar usando os nomes definidos (ex.: ping pong).

6. Configuração necessária

- Instalar ferramentas de rede dentro do contêiner (ex.: `apt-get update && apt-get install iputils-ping -y`).

Dicas e Comandos Importantes

✓ Remover todos os contêineres:

```
docker container rm -f $(docker ps -aq)
```

✓ Definir nome do contêiner:

```
docker run -it --name [NOME] [IMAGEM] [COMANDO]
```

✓ Criar uma rede bridge personalizada:

```
bash
```

```
docker network create --driver bridge [NOME_DA_REDE]
```

✓ **Listar redes disponíveis:**

```
docker network ls
```

✓ **Inspecionar um contêiner (ver redes associadas):**

```
docker inspect [NOME_OU_ID_DO_CONTÊINER]
```

✓ **Testar comunicação entre contêineres:**

```
ping [NOME_DO_OUTRO_CONTÊINER]
```

✓ **Documentação relevante:**

- [Docker: Use bridge networks](#) (resolução automática de DNS em redes user-defined).

Próximos passos: Explorar redes host e none para casos de uso específicos. 🚀

Diferença entre as Redes host e none no Docker

As redes host e none são dois modos de rede especiais no Docker, cada um com um propósito distinto. Vamos explorar suas diferenças e casos de uso:

1. Rede host

Características:

- **Remove o isolamento de rede do contêiner**, fazendo com que ele compartilhe **diretamente** a rede da máquina host (seu sistema operacional).
- O contêiner **não tem seu próprio namespace de rede** – ele usa o IP e as interfaces da máquina host.
- **Portas expostas no contêiner são mapeadas diretamente no host** (não é necessário usar -p para publicar portas).

Quando usar?

- ✓ **Performance máxima** (útil para aplicações sensíveis à latência, como servidores de jogos ou streaming).
- ✓ **Quando você não quer gerenciar NAT ou mapeamento de portas.**
- ✓ **Ferramentas de monitoramento ou redes de alto desempenho** (ex.: Prometheus, Packet sniffers).

Exemplo:

```
docker run --rm --network host nginx
```

- O Nginx estará acessível diretamente na porta 80 do host, sem precisar de -p 80:80.

Desvantagens:

- ✗ **Sem isolamento de rede** (o contêiner pode conflitar com serviços do host).
- ✗ **Inseguro para ambientes multi-tenant** (contêineres podem escutar portas já usadas pelo host).

2. Rede none

Características:

- **Desabilita toda a rede do contêiner** – ele não tem acesso a internet, nem a outros contêineres.
- Útil para **contêineres que não precisam de comunicação externa** (somente processamento local).

Quando usar?

- ✓ **Processamento offline** (ex.: batch jobs, processamento de dados sem saída externa).
- ✓ **Segurança máxima** (contêineres isolados, sem risco de vazamento de dados).
- ✓ **Ambientes restritivos** (ex.: contêineres que só interagem com volumes locais).

Exemplo:

```
docker run --rm --network none alpine sleep 3600
```

- O contêiner não conseguirá fazer ping, acessar a internet ou outros serviços.

Desvantagens:

- ✗ **Sem comunicação externa** (não serve para serviços web ou banco de dados).

Resumo: Comparação entre host e none

Aspecto	Rede host	Rede none
Isolamento	⚠ Nenhum (usa rede do host)	🔒 Total (sem rede)
Performance	⚡ Máxima (sem overhead de rede)	🌲 Inexistente (sem rede)
Uso típico	Serviços de alta performance	Jobs offline / seguros

Aspecto	Rede host	Rede none
Acesso externo	✅ Sim (via host)	❌ Não
Segurança	❌ Baixa (expõe o host)	✅ Alta (totalmente isolado)

Quando escolher cada uma?

- **Use host se:**
 - Precisa de **performance máxima** (ex.: servidores de jogos, streaming).
 - Não se importa com **isolamento de rede**.
- **Use none se:**
 - O contêiner **não precisa de rede** (ex.: processamento de dados offline).
 - Prioriza **segurança e isolamento total**.
- **Use bridge (padrão ou customizada)** na maioria dos casos, pois oferece **isolamento e DNS automático**.

Comandos úteis para redes no Docker

Listar redes disponíveis

```
docker network ls
```

Inspecionar uma rede

```
docker network inspect [nome_da_rede]
```

Criar uma rede bridge personalizada

```
docker network create --driver bridge minha-rede
```

Executar um contêiner em uma rede específica

```
docker run --network [host|none|minha-rede] [imagem]
```

Espero que isso ajude a entender quando usar cada tipo de rede! 🚀

Comunicação entre Contêineres via Rede Docker

Objetivo

Criar dois contêineres (**MongoDB** e **Alura Books**) em uma **rede bridge personalizada** para permitir comunicação via **hostname**.

Passo a Passo

1. Baixar as Imagens

```
docker pull mongo:4.4.6
```

```
docker pull aluradocker/alura-books:1.0
```

2. Criar uma Rede Bridge Personalizada

(Se ainda não existir)

```
docker network create --driver bridge minha-bridge
```

3. Subir o Contêiner do MongoDB

- **Nome do contêiner:** meu-mongo (a aplicação Alura Books busca esse hostname).
- **Rede:** minha-bridge.

```
docker run -d --network minha-bridge --name meu-mongo mongo:4.4.6
```

4. Subir o Contêiner da Aplicação Alura Books

- **Mapeamento de portas:** -p 3000:3000 (expõe a aplicação no host).
- **Rede:** minha-bridge (mesma rede do MongoDB).

```
docker run -d --network minha-bridge --name alurabooks -p 3000:3000 aluradocker/alura-books:1.0
```

5. Testar a Comunicação

1. Acesse no navegador:

text

http://localhost:3000

2. Popular o banco (endpoint /seed):

http://localhost:3000/seed

3. Verifique se os dados aparecem na aplicação.

6. Validar a Comunicação entre Contêineres

- **Parar o MongoDB:**

`docker stop meu-mongo`

- A aplicação **para de funcionar** (dados desaparecem).

- **Reiniciar o MongoDB:**

`docker start meu-mongo`

- Acesse `/seed` novamente e os dados voltam.

Pontos-Chave

✓ Como a Comunicação Funciona?

- **Hostname Resolution:**

- Contêineres na mesma rede **user-defined bridge** (minha-bridge) resolvem nomes via **DNS automático**.
- A aplicação `alura-books` busca o host `meu-mongo` (definido no código).

- **Isolamento vs. Comunicação:**

- `minha-bridge` permite comunicação segura entre contêineres **sem expor portas desnecessárias**.
- `-p 3000:3000` é usado apenas para expor a aplicação ao host.

⚠ Problemas com a Abordagem Manual

- **Ordem de Inicialização:**

- Se o MongoDB não estiver rodando antes do `alura-books`, a aplicação **falha**.

- **Gerenciamento Complexo:**

- Em produção, **não é escalável** subir contêineres manualmente.

✦ Próximos Passos (Sugeridos no Vídeo)

- **Automatizar com docker-compose:**

- Define serviços, redes e dependências em um único arquivo (`docker-compose.yml`).
- Garante que o MongoDB suba **antes** da aplicação.

Comandos Úteis para Debug

Comando	Descrição
<code>docker network ls</code>	Lista redes disponíveis
<code>docker inspect meu-mongo</code>	Verifica se o contêiner está na rede correta
<code>docker logs alurabooks</code>	Checa logs da aplicação em caso de erro
<code>docker network inspect minha-bridge</code>	Mostra contêineres conectados à rede

Conclusão

- **User-defined bridge** é a melhor opção para comunicação estável entre contêineres.
- **Hostnames** são mais confiáveis que IPs dinâmicos.
- **Automatização** (com docker-compose) é essencial para ambientes reais.
- ```
docker run -d --network minha-bridge --name alurabooks -p 3000:3000 aluradocker/alura-books:1.0
```
- ```
docker run -d --network minha-bridge --name meu-mongo mongo:4.4.6
```
- Em seu navegador, acesse a url `localhost:3000` e veja que foi possível carregar a página da aplicação. Para que os dados sejam carregados e armazenados no banco, acesse `localhost:3000/seed` e, em seguida, recarregue a página `localhost:3000`. Veja que as informações agora estão sendo exibidas por conta da comunicação entre aplicação e banco de dados.

Nessa aula aprendemos:

- O docker dispõe por padrão de três redes: bridge, host e none;
- A rede bridge é usada para comunicar containers em um mesmo host;
- Redes bridges criadas manualmente permitem comunicação via hostname;
- A rede host remove o isolamento de rede entre o container e o host;
- A rede none remove a interface de rede do container;
- Podemos criar redes com o comando `docker network create`.



No próximo vídeo: Como simplificar esse processo com docker-compose!

Introdução ao Docker Compose

Problema Atual

- Gerenciar múltiplos contêineres manualmente (docker run, docker stop, docker rm) é **ineficiente** e **propenso a erros**.
- Aplicações complexas exigem **vários serviços** (ex.: banco de dados, back-end, front-end) que precisam ser **coordenados**.

Solução: Docker Compose

- **Ferramenta para definir e executar aplicações multi-contêiner** em um único arquivo YAML.
- **Vantagens:**
 - ✓ **Ordem de inicialização automática** (dependências entre serviços).
 - ✓ **Gerenciamento simplificado** (start/stop com um comando).
 - ✓ **Reprodutibilidade** (mesmo ambiente em qualquer máquina).

Passo a Passo

1. Instalar o Docker Compose (Linux)

(No Windows/Mac, já vem instalado com o Docker Desktop)

Baixar e instalar

```
sudo curl -L "https://github.com/docker/compose/releases/download/v2.23.0/docker-compose-$(uname -s)-$(uname -m)" -o /usr/local/bin/docker-compose
```

Tornar executável

```
sudo chmod +x /usr/local/bin/docker-compose
```

Verificar instalação

```
docker-compose --version
```

2. Criar um Arquivo docker-compose.yml

Exemplo para substituir os comandos manuais do MongoDB e Alura Books:

```
version: '3.8'
```

```
services:
```

mongo: # Nome do serviço (também vira hostname)

image: mongo:4.4.6

networks:

- minha-bridge

Opcional: persistência de dados

volumes:

- mongo_data:/data/db

alura-books:

image: aluradocker/alura-books:1.0

depends_on:

- mongo # Garante que o MongoDB suba primeiro

networks:

- minha-bridge

ports:

- "3000:3000" # Expõe a porta 3000

networks:

minha-bridge:

driver: bridge

volumes:

mongo_data: # Volume para persistir dados do MongoDB

3. Comandos Básicos do Docker Compose

Comando	Descrição
docker-compose up -d	Inicia todos os serviços em background
docker-compose down	Para e remove todos os contêineres/redes

Comando	Descrição
<code>docker-compose logs</code>	Mostra logs dos serviços
<code>docker-compose ps</code>	Lista contêineres em execução

4. Diferença entre Composição e Orquestração

Docker Compose	Orquestradores (Kubernetes, Swarm)
Para ambientes locais/dev	Para produção/cluster
1 host apenas	Multi-hosts (escalabilidade)
Arquivo YAML simples	Configuração complexa (pods, deployments)

Próximos Passos

1. **Testar o** `docker-compose.yml` criado:


```
docker-compose -d
```

 - Acesse `http://localhost:3000/seed` para popular o banco.
2. **Adicionar mais serviços** (ex.: front-end, API).
3. **Explorar recursos avançados:**
 - Variáveis de ambiente (environment).
 - Healthchecks.
 - Configurações de deploy.

Por que Usar Docker Compose?

- **Produtividade:** Elimina comandos repetitivos.
- **Consistência:** Ambiente idêntico para todos os desenvolvedores.
- **Controle Centralizado:** Toda a infraestrutura em um único arquivo.

 **No próximo vídeo:** Como adicionar um **front-end** ao compose!

O Docker Compose realmente resolve o problema de ter que executar múltiplos containers de forma individual e manual. Com o Docker Compose, você pode definir toda a configuração necessária para a sua aplicação em um único arquivo YAML. Dessa forma, você consegue subir todos os containers necessários com apenas um comando, evitando ter que executar vários comandos docker run separadamente.

O Docker Compose facilita muito o gerenciamento de aplicações compostas por múltiplos serviços, pois você pode definir a rede, volumes, variáveis de ambiente e outras configurações de forma centralizada no arquivo de composição. Isso torna o processo de implantação e gerenciamento da sua aplicação muito mais simples e automatizado.

Continue estudando e praticando o Docker Compose, essa ferramenta é muito importante para o desenvolvimento e implantação de aplicações modernas e escaláveis.

Docker Compose - Configuração e Execução

Principais Tópicos

1. **Criação do arquivo docker-compose.yml**
 - Define a estrutura para orquestrar múltiplos containers.
 - Versão utilizada: 3.9.
2. **Definição dos serviços**
 - **MongoDB:**
 - Imagem: mongo:4.4.6
 - Nome do container: meu-mongo
 - Rede: compose-bridge
 - **Alura Books:**
 - Imagem: aluradocker/alura-books:1.0
 - Nome do container: alurabooks
 - Rede: compose-bridge
 - Mapeamento de portas: 3000:3000
3. **Configuração da rede**
 - Rede compose-bridge com driver bridge.
4. **Execução do Docker Compose**
 - Comando docker-compose up para iniciar os containers.

- A aplicação fica acessível em localhost:3000.

5. Encerramento dos containers

- Pressionar Ctrl + C no terminal para parar os serviços.

Principais Comandos

Comando	Descrição
mkdir Desktop/ymls	Cria a pasta ymls no Desktop.
cd Desktop/ymls	Navega até a pasta criada.
code .	Abre o VS Code no diretório atual.
docker-compose up	Inicia os containers conforme o docker-compose.yml.
docker-compose down	Para e remove os containers (não usado neste exemplo, mas útil).

Observação: O Docker Compose simplifica a execução de múltiplos containers com uma única configuração YAML.

[Legacy versions](#) | [Docker Docs](#)

Configurando Dependências e Otimizando Logs no Docker Compose

Principais Tópicos

1. Dependência entre Serviços (depends_on)

- Define a ordem de inicialização dos containers.
- **Observação:** depends_on só garante que o container esteja rodando, **não** que a aplicação dentro dele esteja pronta para receber conexões.

- Exemplo:

alurabooks:

depends_on:

- mongodb

2. Modo Detached (-d)

- Executa os containers em segundo plano, liberando o terminal.
- Comando:

docker-compose up -d

3. Verificação dos Containers (docker-compose ps)

- Lista os serviços em execução de forma organizada.
- Comando:

docker-compose ps

4. Parar e Remover Containers (docker-compose down)

- Encerra os serviços e remove containers, redes e volumes criados.
- Comando:

docker-compose down

5. Configurações Avançadas (deploy e Swarm)

- Configurações como replicas e parallelism só funcionam no modo **Docker Swarm**.
- Exemplo de uso:

docker stack deploy

Principais Comandos

Comando	Descrição
docker-compose up	Inicia os containers conforme o docker-compose.yml.
docker-compose up -d	Inicia os containers em segundo plano.
docker-compose ps	Lista os serviços em execução.
docker-compose down	Para e remove todos os recursos criados.

Comando	Descrição
docker stack deploy (Swarm)	Usado para implantar configurações avançadas (replicas, etc.).

Observações Importantes

- **depends_on ≠ Aplicação Pronta:** Se o Alura Books depende do MongoDB, mas o MongoDB ainda não está aceitando conexões, pode ocorrer erro. Em casos reais, é recomendado implementar **healthchecks** ou lógica de espera na aplicação.
- **Modo Swarm:** Configurações como deploy só funcionam em ambientes Docker Swarm, não em docker-compose comum.
- **Documentação Oficial:** Sempre consulte a [documentação do Docker Compose](#) para configurações específicas (volumes, redes, etc.).

Conclusão: O Docker Compose simplifica a orquestração de múltiplos containers, mas requer atenção em dependências e modos de execução para garantir o funcionamento correto.,

- O Docker Compose é uma ferramenta de coordenação de containers;
- Como instalar o Docker Compose no Linux;
- Como iniciar containers em conjunto com o comando docker-compose up;
- Como criar um arquivo de composição e definir instruções de containers, redes e serviços.

No Windows, os volumes do Docker são armazenados em um caminho diferente devido à arquitetura do sistema e ao uso do Docker Desktop (que roda em uma máquina virtual Linux).

Onde ficam os volumes do Docker no Windows?

Por padrão, os volumes do Docker são armazenados em um **diretório virtual dentro do WSL 2** (Windows Subsystem for Linux), que é gerenciado pelo Docker Desktop.

1. Caminho padrão no Docker Desktop (WSL 2)

Os volumes são armazenados em:

`\\wsl$\docker-desktop-data\data\docker\volumes\`

- Esse caminho só é acessível pelo **Windows Explorer** ou via terminal (`explorer.exe \\wsl$\docker-desktop-data`).

2. Se você estiver usando Docker Toolbox (antigo)

Se estiver usando o **Docker Toolbox** (baseado em VirtualBox), os volumes ficam em:

`C:\Users\<seu-usuário>\.docker\machine\machines\default\`

- Mas essa versão é obsoleta e não recomendada.

Como acessar os volumes no Windows?

Método 1: Via Docker CLI

Você pode listar os volumes e inspecioná-los com:

`docker volume ls` # Lista volumes

`docker volume inspect <nome-do-volume>` # Mostra o Mountpoint (caminho no WSL)

Método 2: Acessando via Windows Explorer

1. Abra o **Windows Explorer** (Win + E).
2. Na barra de endereço, digite:

`\\wsl$\docker-desktop-data\data\docker\volumes\`

3. Navegue até o volume desejado (ex: `_data` contém os arquivos).

Método 3: Usando WSL 2 diretamente

Se você tem o **WSL 2** instalado, pode acessar via terminal:

`wsl -d docker-desktop` # Entra no WSL do Docker

`ls /var/lib/docker/volumes/` # Lista os volumes (caminho Linux)

Observações importantes

1. **Não modifique manualmente** os arquivos dentro de `\\wsl$\\...` sem parar os containers, pois pode corromper dados.
2. Se precisar de **backup ou persistência**, use **bind mounts** (mapeando uma pasta do Windows):

`yaml`

`services:`

`_mongodb:`

`volumes:`

- "C:/meu-docker-data/mongodb:/data/db"

3. Se o Docker Desktop estiver lento, considere mover os dados para outro disco:
 - [Documentação oficial: Disk image location.](#)

✓ Resumo

Localização	Caminho no Windows	Observação
Docker Desktop (WSL 2)	<u>\\wsl\$\docker-desktop-data\data\docker\volumes\</u>	<u>Padrão atual</u>
Docker Toolbox (VirtualBox)	<u>C:\Users\<user>\.docker\machine\...</u>	<u>Obsoleto</u>
Bind Mount (recomendado)	<u>C:/pasta-windows/./caminho-no-container</u>	<u>Melhor para persistência</u>

Se precisar de acesso frequente, **bind mounts** são mais fáceis de gerenciar no Windows.



Resumo da Conversa sobre DevOps, Carreira e Mercado

🚀 Principais Tópicos Abordados

1. O que é DevOps?

- Integração entre **Desenvolvimento (Dev)** e **Operações (Ops)**.
- Foco em **CI/CD** (Integração Contínua/Entrega Contínua), **Infraestrutura como Código (IaC)** e **automatização**.
- Ciclo: Desenvolvimento → Build → Testes → Deploy → Monitoramento.

2. Problemas que o DevOps resolve:

- **Deploys manuais** (erros humanos, rollbacks complexos).
- **Inconsistência entre ambientes** (ex: desenvolvimento vs. produção).
- **Falta de padronização** (soluções como containers e orquestração).

3. Ferramentas Essenciais:

- **CI/CD**: GitHub Actions, GitLab CI, Jenkins.

- **Containers:** Docker, Kubernetes (ou alternativas como AWS ECS).
- **IaC:** Terraform, Ansible.
- **Monitoramento/Testes:** SonarQube, JUnit, PHPUnit.

4. Carreira em DevOps:

- **Não é necessário ser desenvolvedor**, mas ajuda entender código.
- **Certificações em nuvem** (AWS, Azure, GCP) são valiosas.
- **Portfólio no GitHub** com projetos práticos (ex: pipelines CI/CD).

5. Dicas para Iniciantes:

- Comece com **uma ferramenta de cada vez** (ex: GitHub Actions → Terraform).
- Use **versões gratuitas** de nuvens (AWS Free Tier, Azure Credits).
- **Crie vídeos ou tutoriais** mostrando seus projetos (destaque para recrutadores).

6. Faculdade vs. Experiência Prática:

- **Faculdade** traz **base teórica e networking**.
- **Cursos online** (como Alura) e **certificações** complementam a formação.

Principais Comandos e Ferramentas Citados

Categoria	Ferramentas/Comandos
CI/CD	GitHub Actions, GitLab CI, Jenkins
Containers	docker-compose up, kubectl apply -f deployment.yml, aws ecs create-service
IaC	terraform init, terraform apply, ansible-playbook
Monitoramento	sonar-scanner, kubectl get pods, aws cloudwatch alarms

Dicas Finais

- **Não se sobrecarregue:** Aprenda uma ferramenta por vez.
- **Pratique com projetos reais:** Suba um blog usando CI/CD + Docker na AWS.
- **Networking:** Participe de eventos de DevOps e conecte-se no LinkedIn.

- **Cuidado com custos em nuvem:** Configure **budgets e alertas** para evitar surpresas.

"DevOps é sobre cultura, automação e melhoria contínua — não apenas ferramentas."

 Links Úteis:

- [AWS Free Tier](#)
- [Certificação Terraform](#)
- [FIAP - Graduação em Engenharia de Software](#)

Créditos: Fabrício Carraro (Alura), Rubens Rodrigues (FIAP/School Guardian), João Marques (GOP/FIAP).